



Big Data Analytics - BCS604B

“Electronics Inventory & Warranty Tracker

Submitted in partial fulfillment of the requirement for the award of the degree of

Bachelor of Engineering **in** **Computer Science and Engineering**

Submitted By

Chandana H M 1DT23CS042

Deeksha D K 1DT23CS050

Deepika G L 1DT23CS053

Niyathi 1DT23CS059

Under the Guidance of

Prof. Divya HN

Assistant Professor, Department of CSE



Department of Computer Science and Engineering

DAYANANDA SAGAR ACADEMY OF TECHNOLOGY AND MANAGEMENT

(Affiliated to VTU, Belagavi and Approved by AICTE, New Delhi),
CE, CSE, ECE, EEE, ISE, ME Courses Accredited by NBA, New Delhi, NAAC A+

ACADEMIC YEAR 2025-26

ABSTRACT

This project presents a smart Electronics Inventory and Warranty Analytics Dashboard developed using MongoDB, Node.js, Express.js, and Lovable. The system is designed to handle semi-structured and heterogeneous electronics product data using MongoDB's flexible schema capabilities. Different electronic products such as mobiles, laptops, smartwatches, and headphones contain varying specifications. Traditional relational databases struggle to manage such dynamic data structures efficiently. MongoDB solves this problem using document-oriented flexible schemas. The project also provides analytical insights such as inventory health monitoring, low-stock alerts, sales analytics, warranty tracking, category distribution, top-selling products. The dashboard provides real-time analytics and inventory insights including total products, total revenue, low stock alerts, category-wise distribution, monthly sales trends, and warranty expiration notifications. Interactive charts and visualizations improve business monitoring and decision-making processes. This project demonstrates how modern NoSQL databases and AI-assisted frontend generation tools can be integrated to build scalable and flexible retail analytics systems.

INTRODUCTION

Inventory management systems are essential for retail organizations to monitor product availability, sales performance, and stock movement. In electronics retail, products often contain varying and continuously evolving specifications. For example, mobile phones include camera and battery details, laptops contain processor and GPU specifications, while smartwatches may include display and waterproofing information. Traditional relational databases struggle to manage such dynamic structures because they rely on rigid table schemas.

MongoDB, a NoSQL document-oriented database, solves this problem by allowing flexible document structures. This project uses MongoDB to store semi-structured electronic product data efficiently. The frontend dashboard built using Lovable.ai provides an intuitive and futuristic interface to visualize inventory and sales data. The system integrates MongoDB, Node.js, Express.js, React, and Tailwind CSS to create a modern full-stack analytics solution.

The dashboard visualizes inventory insights such as:

- total products
- revenue analytics
- low stock monitoring
- sales trends
- warranty expiry alerts

The project integrates MongoDB, Node.js, Express.js, React, and Tailwind CSS to create a scalable and modern inventory analytics platform.

PROBLEM STATEMENT

Traditional inventory management systems based on relational databases face multiple challenges while handling semi-structured electronics product data. Different products require different specifications and attributes, making fixed schemas difficult to maintain.

The major problems include:

- rigid schema design
- frequent schema modifications
- poor scalability
- complex joins for analytics
- difficulty handling heterogeneous product specifications

Electronics products such as mobiles, laptops, smartwatches, and headphones continuously evolve with new specifications and features. Therefore, there is a need for a flexible inventory management system capable of:

- storing varying product structures
- providing scalable storage
- generating real-time analytics
- monitoring warranty and inventory health

This project addresses these issues using MongoDB flexible schemas and interactive analytics dashboards.

OBJECTIVES

The objectives of the proposed project are:

1. To design a flexible NoSQL schema for storing electronics inventory data.
2. To demonstrate MongoDB document-based storage for heterogeneous product specifications.
3. To develop backend REST APIs using Node.js and Express.js.
4. To build an interactive dashboard using Lovable.ai and React.
5. To visualize inventory health and sales analytics.
6. To monitor warranty expiration information.
7. To demonstrate the advantages of NoSQL databases over relational systems.
8. To improve business decision-making through real-time analytics.

EXISTING SYSTEM

Existing inventory management systems mainly use relational databases such as MySQL and Oracle. These systems require predefined tables and fixed schemas. Handling products with varying specifications becomes difficult because every new field requires schema modifications.

For example:

- mobiles require battery and camera details
- laptops require processor and GPU information
- smartwatches require display and waterproofing details

Managing all these using relational tables increases complexity.

Limitations of Existing Systems

- rigid schema structure
 - difficult schema updates
 - poor support for semi-structured data
 - reduced scalability
 - complex database joins
 - inefficient handling of dynamic product specification
- These limitations motivated the development of a flexible NoSQL-based inventory analytics system.

PROPOSED SYSTEM

The proposed system uses MongoDB as a flexible document-based database to efficiently manage electronic inventory data. Backend APIs are developed using Node.js and Express.js to communicate between the frontend dashboard and MongoDB.

The frontend dashboard generated using Lovable provides:

- KPI cards
- inventory monitoring
- sales analytics
- category distribution charts
- warranty tracking
- search and filter functionality
- responsive user interface

The system supports flexible schemas where different products can contain different specifications within the same collection.

SYSTEM ARCHITECTURE

The architecture of the system consists of three major layers:

1. Database Layer

MongoDB stores:

- product documents
- sales data
- warranty information

using flexible schemas.

2. Backend Layer

Node.js and Express.js provide REST APIs for:

- retrieving products

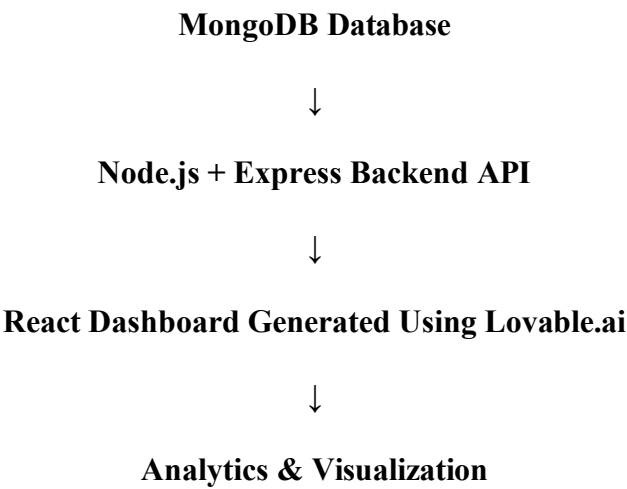
- fetching sales analytics
- inventory management

3. Frontend Layer

Lovable.ai generates a React-based dashboard for:

- analytics visualization
- charts
- inventory monitoring
- interactive UI components

4. Architecture Flow



TECHNOLOGIES USED

TECHNOLOGY	PURPOSE
MongoDB	Flexible NoSQL database
Node.js	Backend runtime environment
Express.js	REST API development
React.js	Frontend UI framework
Lovable	AI-powered dashboard generation
Tailwind CSS	Responsive styling
Recharts	Data visualization and charts

DATABASE DESIGN

Database Name

ElectronicsInventoryDB

Collections

1. products
2. sales

The *products* collection stores inventory and specification details of electronic products.

Example fields:

- name
- category
- brand
- price
- stock
- specifications
- warranty

MongoDB flexible schemas allow storing different product specifications inside the same collection.

Example

- laptops contain processor and GPU details
- mobiles contain camera and battery details
- headphones contain wireless and noise cancellation features

The *sales* collection stores:

- product name
- quantity sold
- revenue

- sales region
- sale date

MONGODB QUERIES

Low Stock Products

```
db.products.find({
  stock: { $lt: 20 }
})
```

Total Revenue

```
db.sales.aggregate([
{
  $group: {
    _id: null,
    totalRevenue: {
      $sum: "$revenue"
    }
  }
}]
```

Category Distribution

```
db.products.aggregate([
{
  $group: {
    _id: "$category",
    count: { $sum: 1 }
  }
}]
```

Warranty Expiry

```
db.products.find({
  "warranty.expiry_date": {
    $lt: "2027-01-01"
  }
})
```


IMPLEMENTATION

The backend server was implemented using Node.js and Express.js. Mongoose was used to connect MongoDB with the backend application. Flexible schemas were implemented using:

strict: false

This allowed storing dynamic product specifications efficiently.

The frontend dashboard was generated using Lovable.ai and customized using React and Tailwind CSS. The dashboard fetches product and sales data from backend APIs and visualizes analytics using charts and KPI cards.

Implemented Features

- product analytics dashboard
- inventory monitoring
- revenue tracking
- low stock alerts
- warranty expiry notifications
- category-wise product distribution
- responsive dashboard UI

RESULTS

The system was tested successfully with multiple electronics product categories including:

- mobiles
- laptops
- smartwatches
- headphones

MongoDB efficiently stored heterogeneous product specifications within the same collection.

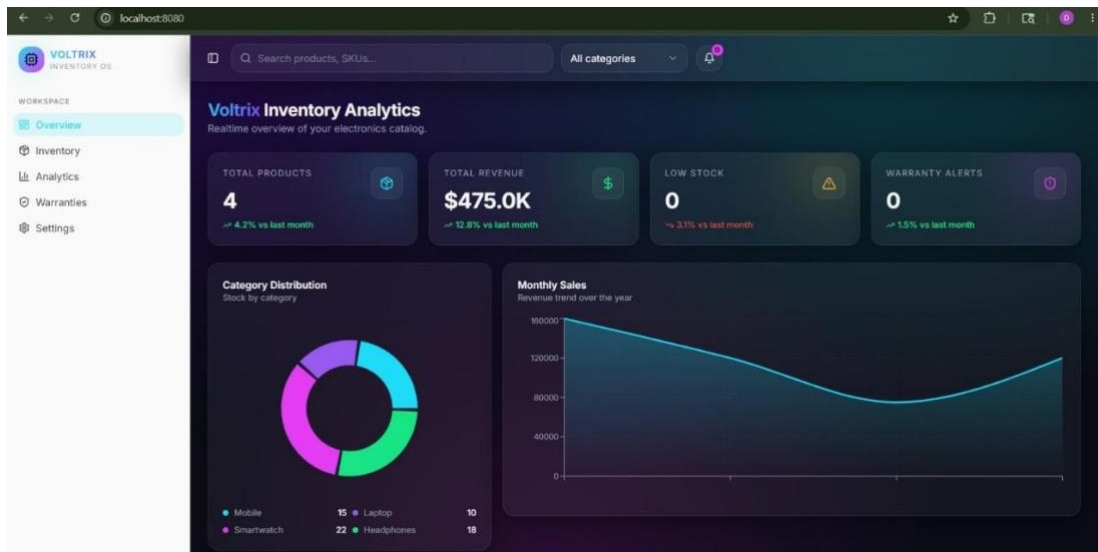
The dashboard successfully displayed:

- total products
- total revenue

- sales analytics
- category distribution
- low stock alerts
- warranty expiry monitoring

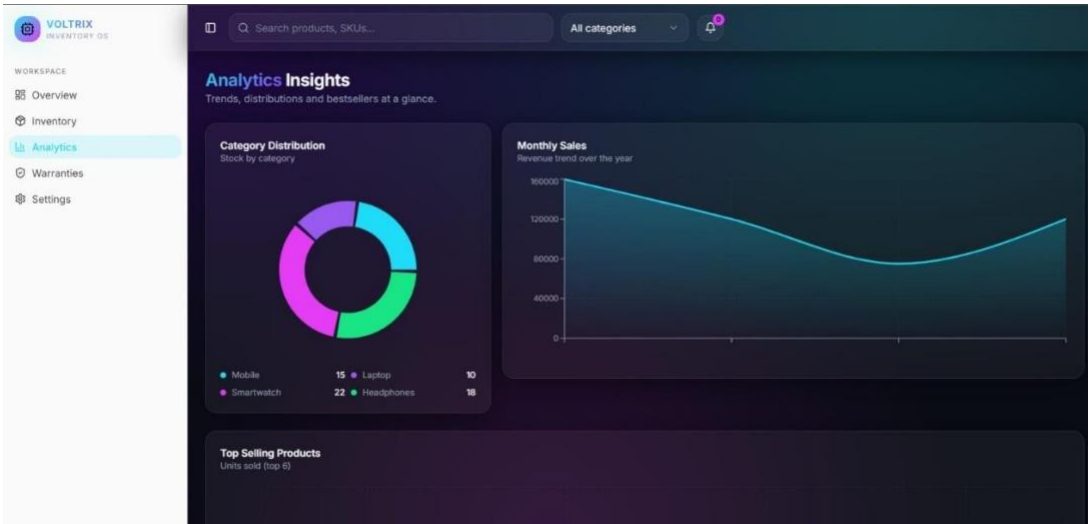
The project demonstrated successful integration between MongoDB, backend APIs, and React dashboard components.

SCREENSHOTS:



The screenshot displays the 'Inventory Catalog' section of the dashboard. It features a sidebar with navigation links: Overview, Inventory, Analytics, Warranties, and Settings. The main content area includes a search bar, a category filter set to 'All categories', and a notification bell. Below these, there is a table titled 'Inventory' with 4 items. The table has columns for PRODUCT, CATEGORY, PRICE, and STOCK STATUS.

PRODUCT	CATEGORY	PRICE	STOCK STATUS
iPhone 15	Mobile	\$79,999	15 In stock
Dell XPS 15	Laptop	\$120,000	10 In stock
Galaxy Watch 7	Smartwatch	\$24,999	22 In stock
Sony WH-1000XM5	Headphones	\$29,999	18 In stock



VOLTRIX INVENTORY OS

WORKSPACE

- Overview
- Inventory
- Analytics
- Warranties**
- Settings

Search products, SKUs... All categories

Warranty Tracking

0 items expiring within 30 days.

Warranty Expirations			Sorted by urgency
PRODUCT	EXPIRES	DAYS STATUS	
iPhone 15		NaN	Active
Dell XPS 15		NaN	Active
Galaxy Watch 7		NaN	Active
Sony WH-1000XM5		NaN	Active

VOLTRIX INVENTORY OS

WORKSPACE

- Overview
- Inventory
- Analytics
- Warranties
- Settings**

Search products, SKUs... All categories

Settings

Configure your workspace preferences.

Data Source	http://localhost:5000
Theme	Futuristic Dark
Refresh Interval	Realtime
Notifications	Enabled

products +

localhost:27017 > ElectronicsInventoryDB > products [Open MongoDB shell](#)

Documents 4 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#) ✚ Explain Reset Find Options ▶

ADD DATA UPDATE DELETE EXPORT DATA EXPORT CODE 25 1 - 4 of 4

```
{
  "_id": ObjectId("69fb45dc60a0efec36b8b85"),
  "name": "iPhone 15",
  "category": "Mobile",
  "brand": "Apple",
  "price": 79999,
  "stock": 15,
  "specifications": Object,
  "warranty": Object
}
```

```
{
  "_id": ObjectId("69fb45dc60a0efec36b8b87"),
  "name": "Dell XPS 15",
  "category": "Laptop",
  "brand": "Dell",
  "price": 120000,
  "stock": 10,
  "specifications": Object,
  "warranty": Object
}
```

```
{
  "_id": ObjectId("69fb45dc60a0efec36b8b89"),
  "name": "Galaxy Watch 7",
  "category": "Smartwatch",
  "brand": "Samsung",
  "price": 24999,
  "stock": 22,
  "specifications": Object,
  "warranty": Object
}
```

sales +

localhost:27017 > ElectronicsInventoryDB > sales [Open MongoDB shell](#)

Documents 4 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#) ✚ Explain Reset Find Options ▶

ADD DATA UPDATE DELETE EXPORT DATA EXPORT CODE 25 1 - 4 of 4

```
{
  "_id": ObjectId("69fb4615c60a0efec36b8b8e"),
  "product_name": "iPhone 15",
  "category": "Mobile",
  "quantity": 2,
  "revenue": 159998,
  "sale_date": "2026-05",
  "region": "Bangalore"
}
```

```
{
  "_id": ObjectId("69fb4620c60a0efec36b8b90"),
  "product_name": "Dell XPS 15",
  "category": "Laptop",
  "quantity": 1,
  "revenue": 120000,
  "sale_date": "2026-05",
  "region": "Mumbai"
}
```

```
{
  "_id": ObjectId("69fb4629c60a0efec36b8b92"),
  "product_name": "Galaxy Watch 7",
  "category": "Smartwatch",
  "quantity": 3,
  "revenue": 74997,
  "sale_date": "2026-05",
  "region": "Delhi"
}
```

CONCLUSION

The Electronics Inventory and Warranty Analytics Dashboard successfully demonstrates the implementation of a modern full-stack inventory management and analytics system using MongoDB, Node.js, Express.js, React, and Lovable. The project was developed to address the limitations of traditional relational database systems in handling heterogeneous and semi-structured electronics product data.

MongoDB's flexible schema model proved highly effective for storing different product categories such as mobiles, laptops, smartwatches, and headphones within the same collection, even though each product contains different specifications and attributes. This flexibility reduced schema complexity and improved scalability compared to conventional SQL-based inventory systems.

The backend APIs developed using Node.js and Express.js enabled efficient communication between the frontend dashboard and MongoDB database. The React-based dashboard generated using Lovable.ai provided an interactive and user-friendly interface for visualizing inventory health, revenue analytics, category distribution, warranty alerts, and sales trends through charts, KPI cards, and tables.

The developed system demonstrated the advantages of NoSQL databases in modern retail and analytics applications where data structures continuously evolve. The project also highlighted how AI-assisted frontend generation tools like Lovable.ai can significantly speed up the development of professional dashboard interfaces.

Overall, the Electronics Inventory and Warranty Analytics Dashboard provides a scalable, efficient, and modern solution for inventory management and analytics. The system can be further enhanced with AI-based forecasting, cloud deployment, authentication mechanisms, barcode integration, and real-time notifications, making it suitable for real-world enterprise-level inventory management applications in the future.